Let’s examine the code in the `MPC-app` that does this hashing and signing. For the micropayment message in the MPC-Dapp, you’ll consider only the amount and the smart contract address. The following code snippets from app.js do the magic--hash the message, and sign it:

```
constructPaymentMessage:function(contractAddress, weiamount) {
      return App.web3.utils.solioditySha3(contractAddress, weiamount)
    }
...
web3.personal.sign(message, web3.eth.defaultAccount, function(err,
                                      signedMessage)
```

Open app.js from the codebase for MPC (in the MPC-app/src/js/ directory) in any editor-reader, such as Sublime or Atom. The first snippet shown here is a `constructPaymentMessage()` for hashing the message elements; the second snippet does the signing and is from the `signMessage ()` function.

You can observe the use of web3 to access the blockchain functions in the web3 API. The `web3.utils` package is used to call the hash function `SolioditySha3` (*Sha* means *secure hash*), and the `web3.personal` package is used to call the function sign that encrypts the hash computed with the private key of the account. Review the parameters of the sign function. The first parameter is the hashed message to be signed, the second parameter is the account number whose private key is used in signing the message, and the third parameter is a callback function. The signed message or an error is returned through this callback function. This code snippet is general enough. Open the src/js/app.js from the MPC-app directory, and examine the functions `constructPaymentMessage()` and `signMessage()`. You can reuse these functions and patterns in other Ethereum-based applications that need secure digital signing.